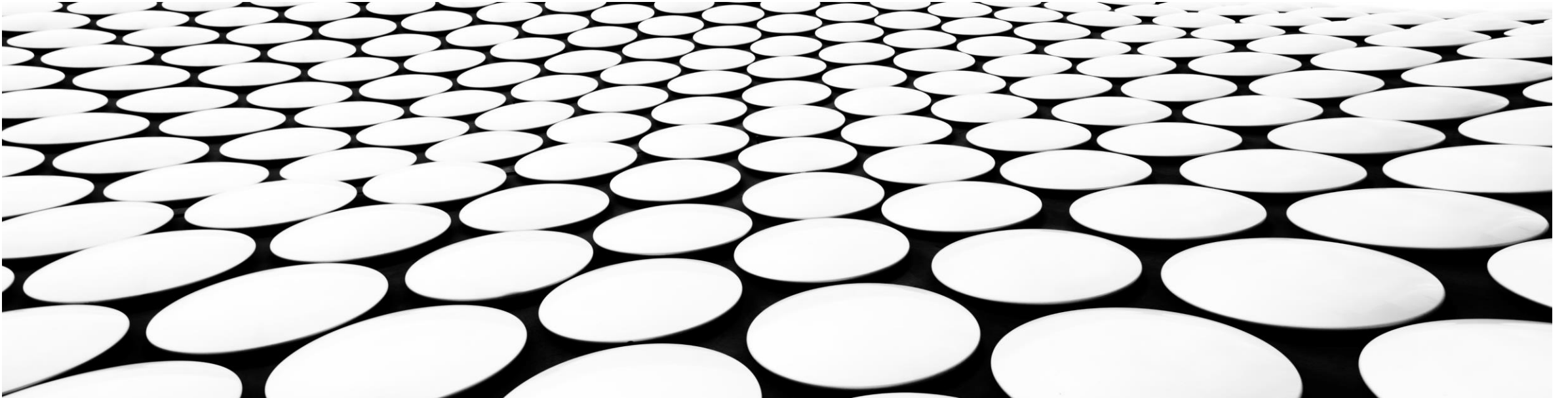

DATA STRUCTURE

LAB1: POINTERS



INDIRECTION

- Pointer variables contain memory addresses as their values.
- Normally, a variable directly contains a specific value, A pointer contains the memory address of a variable that in turn contains a specific value.
- a variable name directly references a value, and a pointer indirectly references a value.
- Referencing a value through a pointer is called *Indirection*.

DIRECT AND INDIRECT REFERENCING

count



count directly
reference a variable
that contains a value 7

countPtr



count



Pointer **countPtr**
indirectly reference a
variable that contains
a value 7

Fig 1.1 ■

DECLARING POINTERS

- Pointers like any variables must be declared before they can be used.
- Example: `int countPtr;`
`double *xPtr, *yPtr;`

POINTER OPERATORS

➤ ADDRESS (&) OPERATOR:

- The address operator (&) is a unary operator that obtains the memory address of its operand.

- Example:

```
int y = 5 ;           //declare variable

int *yPtr = nullptr;  //declare and initialize pointer

yPtr = &y;            // assign address of y to pointer yPtr
```

POINTER OPERATORS

➤ INDIRECTION (*) OPERATOR:

- The indirection or dereferencing operator (*) is a unary operator that returns the value of the object to which it points .
- Example:

```
cout << *yPtr <<endl ;    //display the value of variable y indirectly which is 5
```

```
cout << y<<endl ;          //display the value of variable y directly
```

```
*yPtr = 9;                 // assign 9 to variable y
```

```
cin >> *yPtr;              // receive input value to y by dereferencing a pointer
```

POINTER OPERATORS

REPRESENTING OF A POINTER POINTING TO A VARIABLE IN MEMORY:

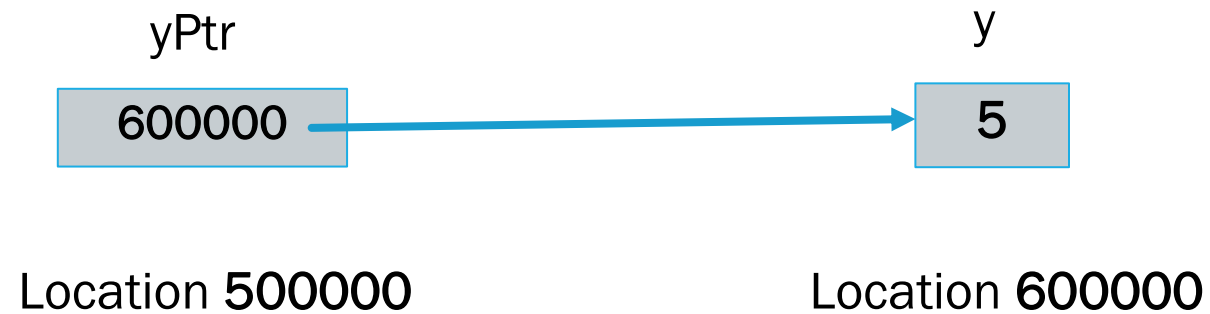


Fig 1.2 ■

USING THE ADDRESS (&) AND INDIRECTION (*) OPERATORS

```
#include <iostream>

using namespace std;

int main()
{
    int a = 7;
    int *aPtr = &a;
    cout<<"The address of a is: "<<&a<<endl;
    cout<<"The address of a is: "<<aPtr<<endl;
    cout<<"The value of a is: "<<a<<endl;
    cout<<"The value of a is: "<<*aPtr<<endl;

    return 0;
}
```

```
The address of a is: 0x6afee8
The address of a is: 0x6afee8
The value of a is: 7
The value of a is: 7
```

PASS BY VALUE

```
#include <iostream>
using namespace std;

int cubeByValue(int);    //prototype

int main()
{
    int number = 5;
    cout<<"The original value of number is: "<<number<<endl;
    number = cubeByValue(number);
    cout<<"The new value number is: "<<number<<endl;
    return 0;
}

int cubeByValue(int n)
{
    return n*n*n;
}
```

The original value of number is: 5
The new value number is: 125

PASS BY REFERENCE WITH POINTERS

```
#include <iostream>
using namespace std;

int cubeByReference(int *);    //prototype

int main()
{
    int number = 3;
    cout<<"The original value of number is: "<<number<<endl;
    number = cubeByReference(&number);
    cout<<"The new value number is: "<<number<<endl;
    return 0;
}

int cubeByReference(int *nPtr)
{
    return *nPtr * *nPtr * *nPtr;
}
```

The original value of number is: 3
The new value number is: 27

EXERCISE

- Create a function to calculate (log) base-10 of user input number using pass-by-reference method, display the original value and the result.
- Create an array of 5 elements and display the addresses of all elements.

REFERENCES

- Mark Weiss, (Data Structures & Algorithm Analysis in C++).
- Narasimha Karumanchi , (Data Structures and Algorithms Made Easy C).